

OAT 1– Docker,Docker–Compose e K8s

Autor: Felipe Souza Teixeira da Silva
Fernando Gustavo Barbosa Santos
Gustavo dos Santos Nobre
Joadson Breno Neves Pereira
João Paulo Rosa Batista

Agenda

O objetivo dessa apresentação é demonstrar a modernização da infraestrutura da **MecâniQA Tech**, utilizando containerização, orquestração e Infraestrutura como Código para criar um ambiente mais organizado, escalável e resiliente

1. Docker e Containerização

Empacotamento da aplicação Java, MySQL e Redis em containers independentes.

2. Docker Compose e Redes

Orquestração dos serviços, comunicação entre containers e persistência dos dados.

3. Kubernetes

Gerenciamento da aplicação através de Pods, Deployments e Services.

4. Terraform e IaC

Definição e provisionamento da infraestrutura utilizando código.

5. Validação e Entrega

Testes do ambiente, monitoramento e versionamento dos artefatos no repositório.

Quem somos nós?



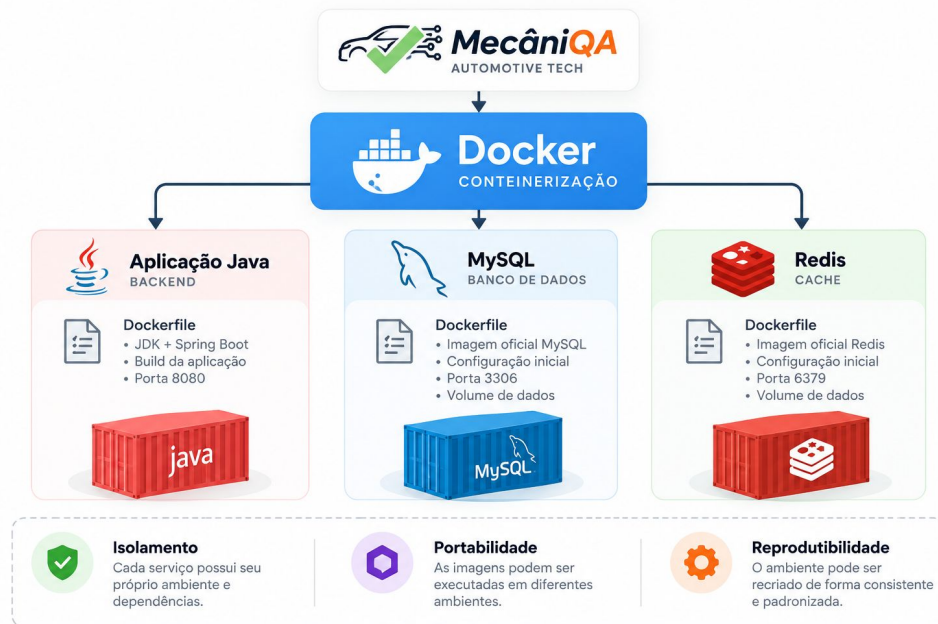
- A **MecâniQA Automotive Tech** é uma startup que oferece soluções de gestão para lojas de autopeças, auto centers e oficinas mecânicas.
- Nosso cliente nos veio com uma questão falando que com o crescimento da operação, a infraestrutura local passou a apresentar **dificuldades de escalabilidade e esgotamento frequente de recursos**, tornando necessário modernizar a forma como os serviços eram executados e gerenciados.

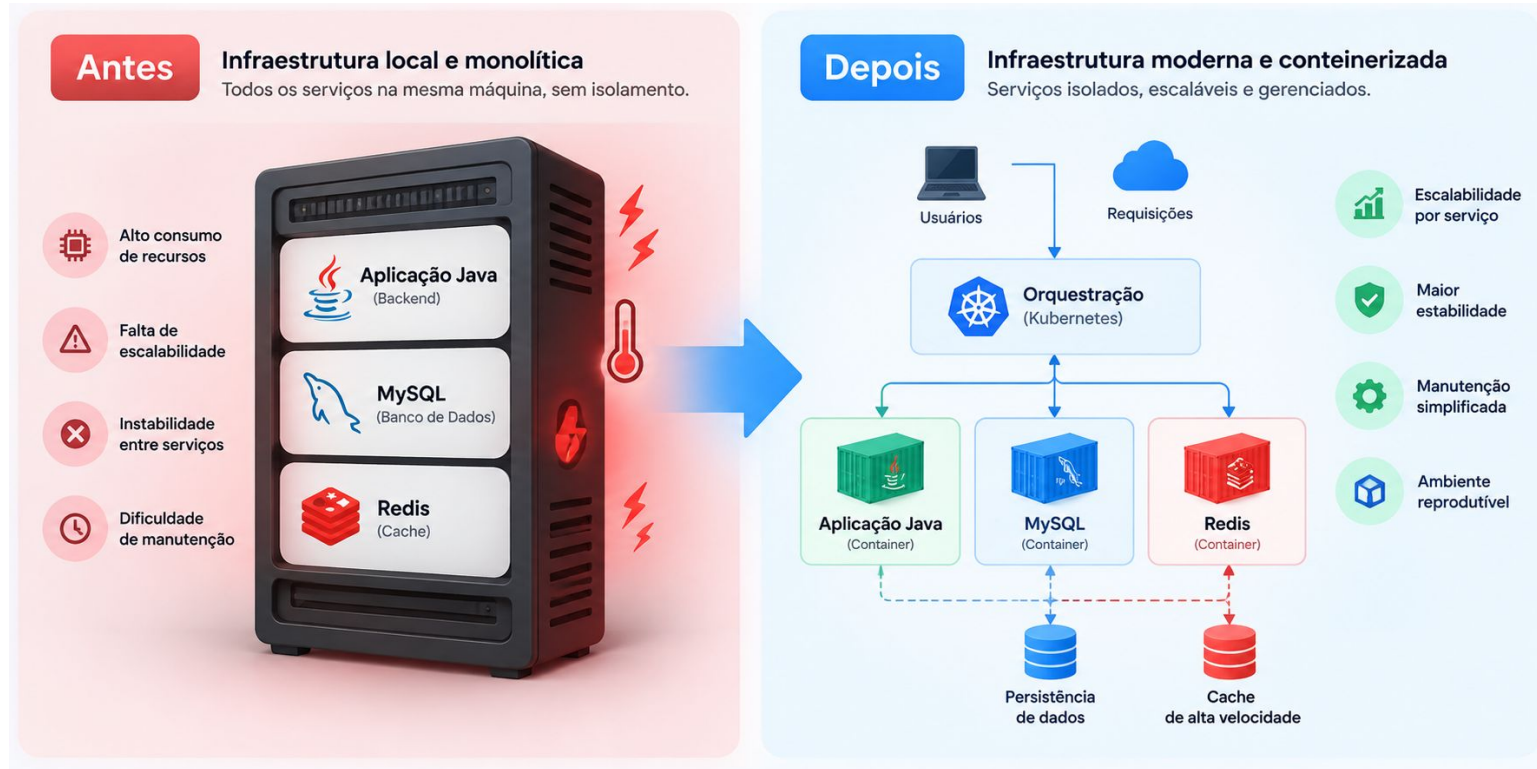


Docker e Containerização

Para iniciar a modernização da infraestrutura da **MecâniQA**, utilizamos o **Docker** para containerização dos principais serviços da aplicação.

Com a containerização, cada serviço passa a possuir seu próprio ambiente de execução e suas dependências, criando a base necessária para posteriormente realizar a **orquestração com Docker Compose e Kubernetes**.





Docker Compose e Redes



Após a containerização dos serviços, utilizamos o Docker Compose para executar e integrar toda a aplicação da MecâniQA em um único ambiente.

O arquivo docker-compose.yml organiza os três serviços — **Backend Java, MySQL e Redis** — conectando-os através da rede mecaniqa-network.

Também configuramos volumes persistentes para MySQL e Redis, evitando a perda de dados quando os containers são reiniciados e dependências para garantir que o backend seja iniciado somente quando os serviços necessários estiverem disponíveis.

```

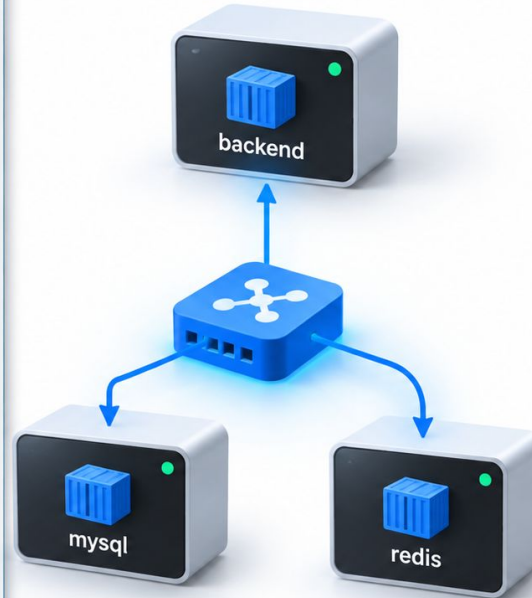
C:\Windows\System32\cmd.exe
C:\projects\mecaniga>docker compose up -d
[+] Running 3/3
✓ Network mecaniga-network    Created
✓ Container mysql             Started
✓ Container redis              Started
✓ Container backend            Started

C:\projects\mecaniga>docker ps
CONTAINER ID   IMAGE      COMMAND                  STATUS    PORTS
a1b2c3d4e5f6   backend:latest  "python app.py"         Up 5 seconds    0.0.0.0:8000->8000/tcp
b2c3d4e5f6a7   mysql:8.0      "docker-entrypoint..." Up 5 seconds    3306/tcp
c3d4e5f6a7b8   redis:7.2      "docker-entrypoint..." Up 5 seconds    6379/tcp

C:\projects\mecaniga>docker network ls
NETWORK ID     NAME      DRIVER  SCOPE
f1e2d3c4b5a6   bridge    bridge  local
a4b3c2d1e6f5   host      host    local
d6e5f4a3b2c1   mecaniga-network  bridge  local

C:\projects\mecaniga>docker inspect mecaniga-network
[
  {
    "Name": "mecaniga-network",
    "Id": "d6e5f4a3b2c1e7f8d9a0b1c2d3e4f567890abcd1234567890abcdef123456",
    "Driver": "bridge",
    "Scope": "local",
    "Containers": {
      "a1b2c3d4e5f6": {
        "Name": "backend",
        "IPv4Address": "172.20.0.3/16"
      },
      "b2c3d4e5f6a7": {
        "Name": "mysql",
        "IPv4Address": "172.20.0.2/16"
      },
      "c3d4e5f6a7b8": {
        "Name": "redis",
        "IPv4Address": "172.20.0.4/16"
      }
    }
  }
]
C:\projects\mecaniga>

```



Kubernetes



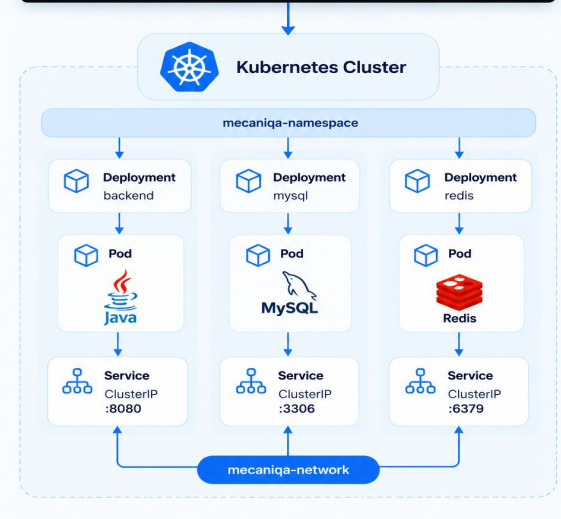
Após estruturar os containers com Docker e Docker Compose, avançamos para o **Kubernetes**, responsável por gerenciar e orquestrar os serviços da aplicação de forma declarativa.

Criamos manifestos para o **Backend Java**, **MySQL** e **Redis**, utilizando recursos como **Deployments** e **Services** para controlar a execução e a comunicação entre os componentes.

```
C:\Windows\System32\cmd.exe
C:\mecanika> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-7d6c9f8b9d                  1/1     Running   0           2m
mysql-6f9d8c7b54                    1/1     Running   0           2m
redis-5c7f6d8b6c                    1/1     Running   0           2m

C:\mecanika> kubectl get services
NAME    TYPE        CLUSTER-IP   PORT(S)    AGE
backend ClusterIP    10.96.12.34   8080/TCP   2m
mysql   ClusterIP    10.96.85.21   3306/TCP   2m
redis   ClusterIP    10.96.47.18   6379/TCP   2m

C:\mecanika> |
```

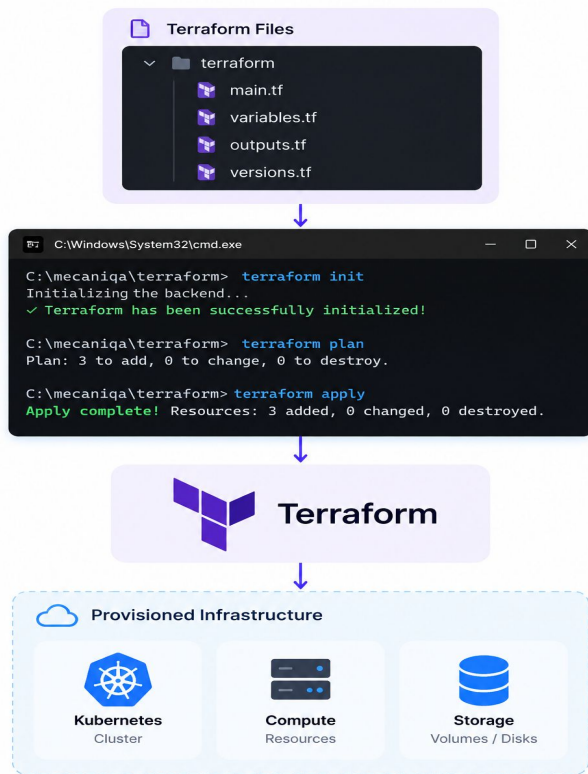


Terraform e IaC



Para complementar a modernização da infraestrutura, utilizamos o **Terraform** aplicando o conceito de **Infrastructure as Code (IaC)**.

A infraestrutura passa a ser definida através de **arquivos de código**, em vez de depender exclusivamente de configurações e ações manuais. No projeto, organizamos essa estrutura utilizando arquivos como **main.tf**, **variables.tf**, **versions.tf** e **outputs.tf**.



Validação e Entrega Final



Após a implementação, realizamos a **validação da infraestrutura** para verificar a integração e o funcionamento dos serviços da MecâniQA nos diferentes ambientes configurados.

Foram verificadas a execução dos **containers**, a comunicação entre **Backend, MySQL e Redis**, a persistência dos dados e a estrutura de orquestração definida para **Docker Compose e Kubernetes**.



A entrega foi organizada e versionada no **GitHub**, reunindo os Dockerfiles, **docker-compose.yml**, manifestos Kubernetes, arquivos Terraform e a documentação necessária para **reproduzir e evoluir o ambiente**.

Considerações finais



Antes



Aplicação no servidor físico

Todos os serviços executando diretamente na mesma máquina.



Configurações manuais

Processo de instalação e configuração manual, sujeito a erros e inconsistências.



Limitações de infraestrutura

Dificuldade para isolar serviços e escalar quando necessário.



Depois



Terraform

Infraestrutura como Código (IaC)



Kubernetes

Orquestração e gerenciamento dos serviços



Backend

Docker



MySQL

Containerizado



Redis

Containerizado



Tempo de processamento

Com a separação dos serviços e melhor uso dos recursos, a aplicação tende a processar requisições de forma mais eficiente, reduzindo tempos de resposta.



Tempo de implantação

A infraestrutura pode ser criada e configurada de forma automatizada com o Terraform, reduzindo o tempo de setup e facilitando a replicação em diferentes ambientes.



Uso de recursos

Cada serviço roda em seu próprio container e é gerenciado pelo Kubernetes, permitindo um uso mais eficiente de CPU e memória e evitando que um serviço afete os outros.



Estabilidade e manutenção

O ambiente fica mais estável e fácil de manter, com configurações padronizadas, versionadas e reproduzíveis, reduzindo falhas e inconsistências.

Referências

- <https://www.hostinger.com/br/tutoriais/o-que-e-docker/>
- <https://medium.com/@habbema/docker-compose-8c16d02585b4>
- <https://developer.hashicorp.com/terraform/tutorials>
- <https://cloud.google.com/learn/what-is-kubernetes?hl=pt-BR>